

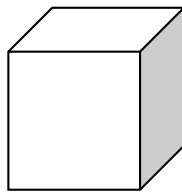
Shadow Computation

Contents

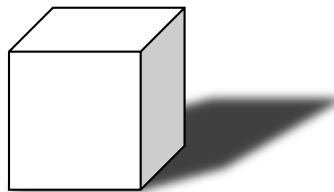
- ❑ Shadows projected onto the ground plane
- ❑ Shadow scanline algorithm
- ❑ Shadow volumes
- ❑ Derivation of shadow polygons from light source transformations
- ❑ Shadow Z-Buffer

Introduction

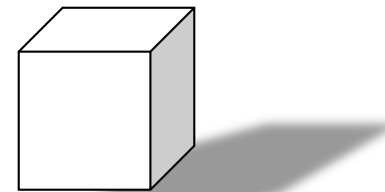
- ❑ Shadow computation is one of the most common approach for photorealistic presentation
- ❑ Shadows add information in the graphics presentation: relative position, light source position, geometry
- ❑ Shadow computation means the computation of the shadow polygons
- ❑ Issues:
 - Type of light source: point, directional (spotlight, area light source)
 - Type of shadow: soft shadow, hard shadow
 - Type of projection: on ground plane, onto any plane, on neighbor objects
 - Computation approach: empirical, procedural, local or global model, etc.



(a)

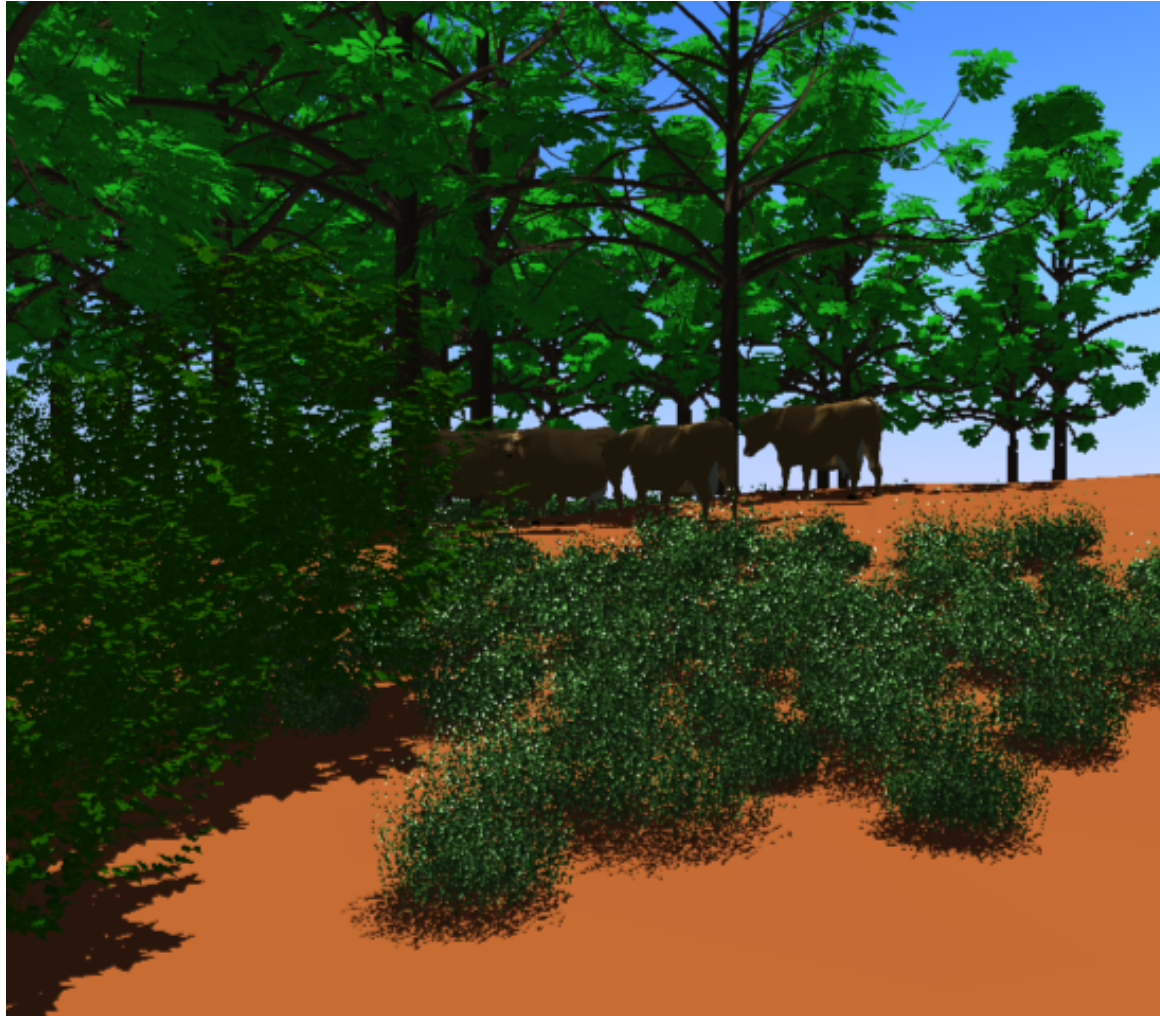


(b)

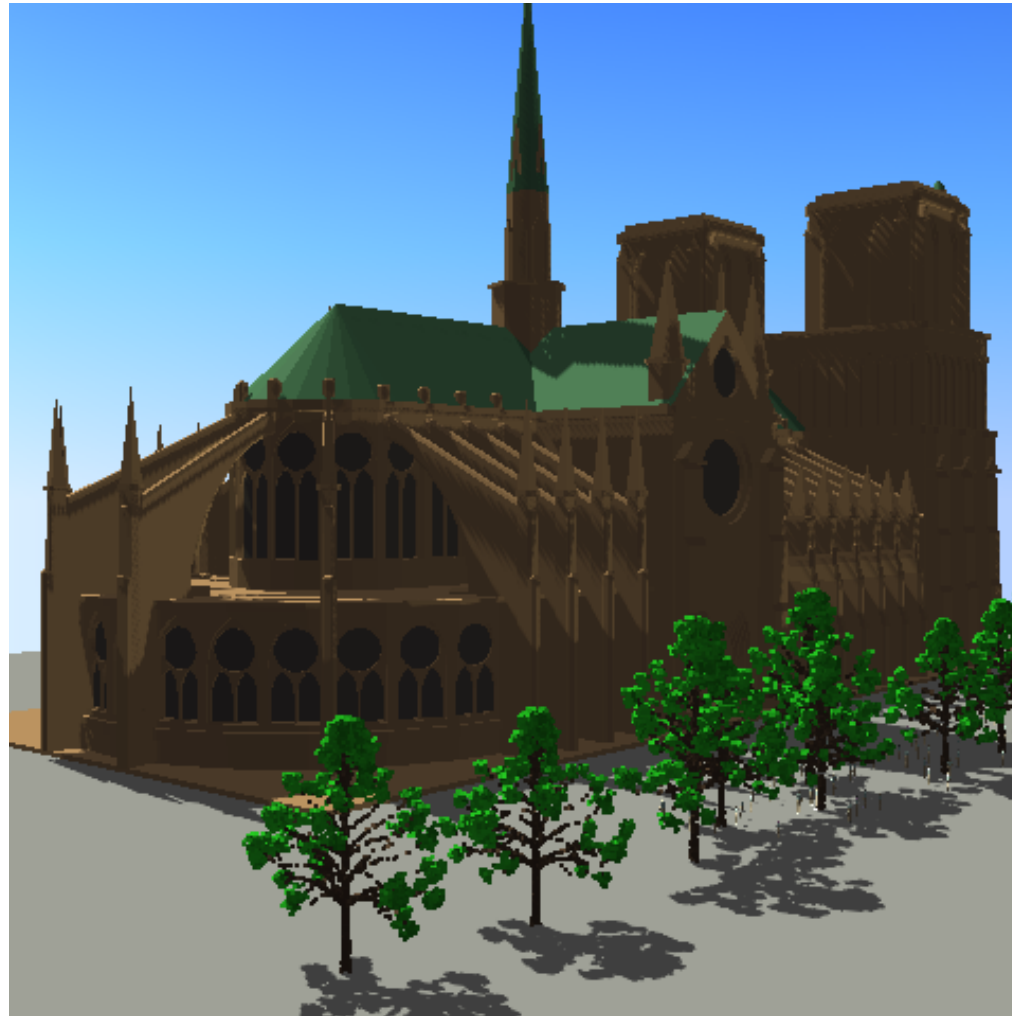


(c)

Shadows



Shadows

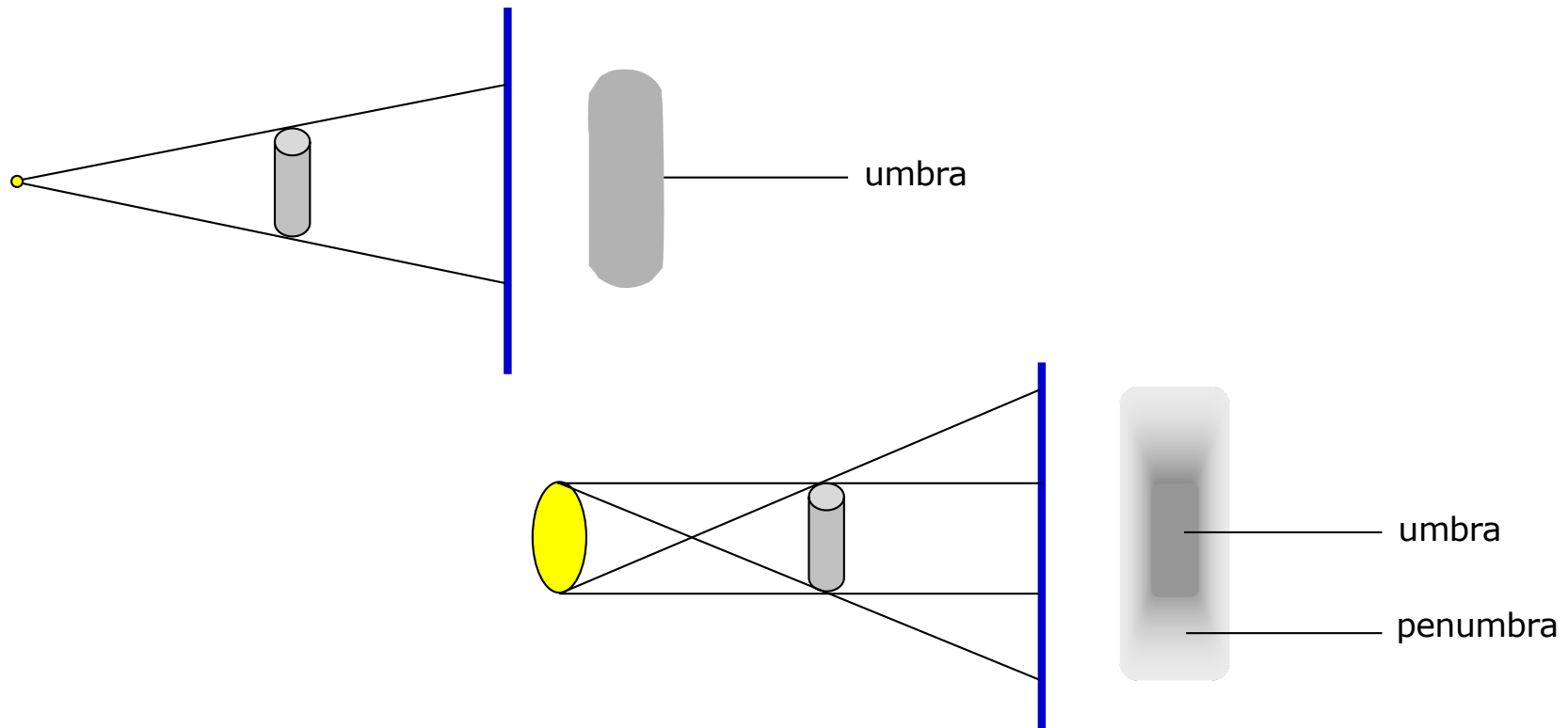


Shadows

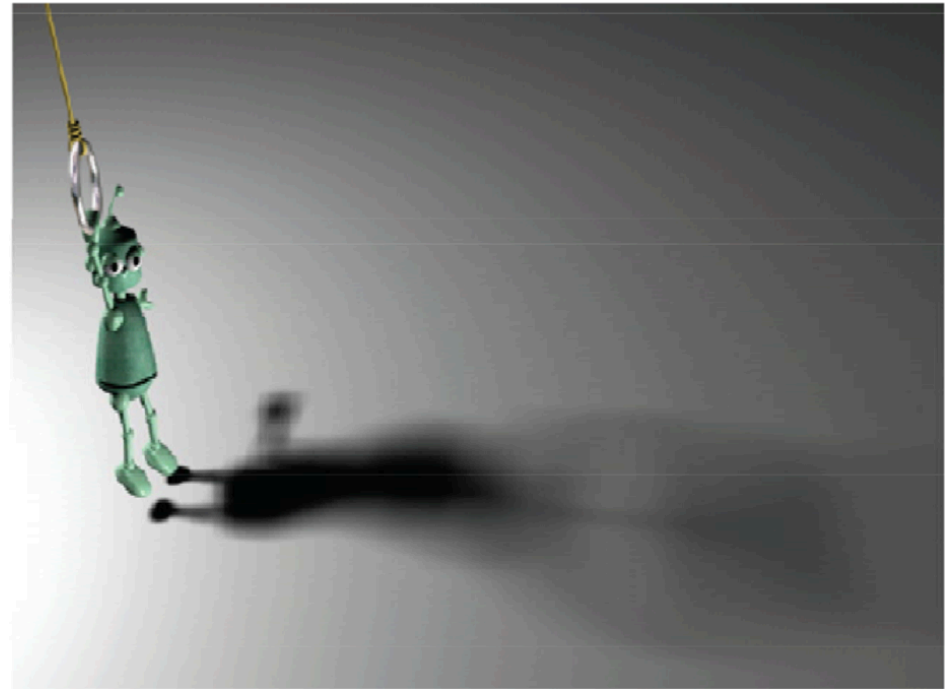
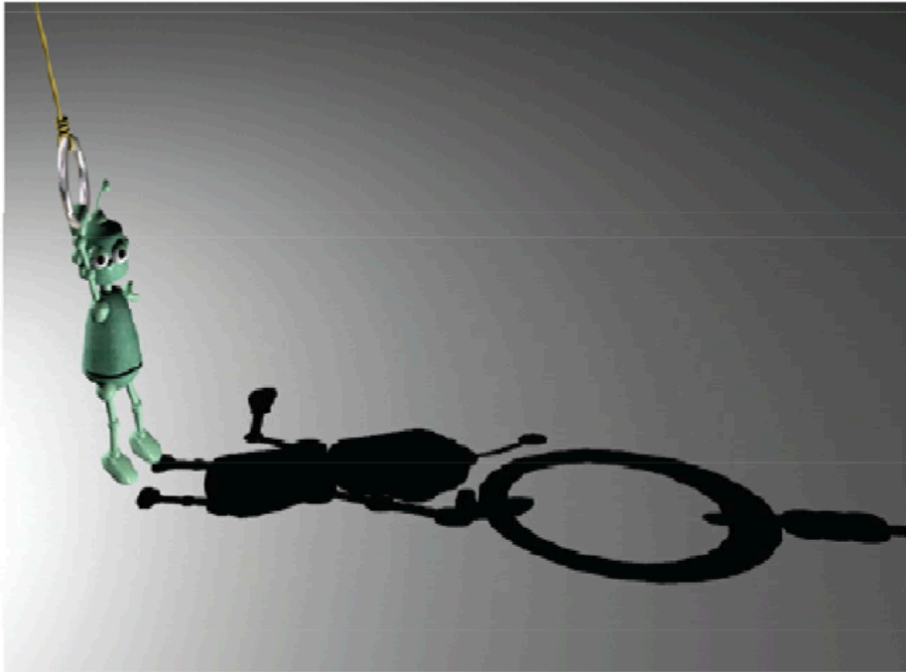


Shadows – umbra and penumbra

- ❑ Umbra – part of the shadow that is completely out off from the light source
- ❑ Penumbra – area that receives some light from the light source and is a gradual change in intensity from umbra to penumbra



Hard and soft shadows



Shadow computation algorithms

□ Global illumination model

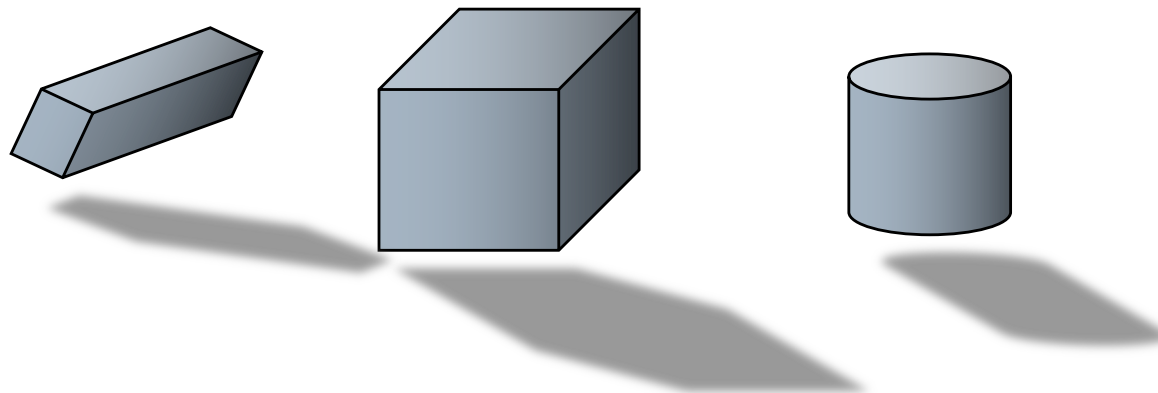
1. Ray tracing
2. Radiosity

□ Local illumination model

1. Shadows projected onto the ground plane
 - Create shadow polygon for each object polygon
2. Scan line algorithm integrates the shadow polygons
 - Shadows are computed through the image rendering by hidden surfaces
3. Shadow volumes
 - Compute shadow volumes for each source light and analyze for each polygon the inside and outside location against the shadow volume
4. Derivation of shadow polygons from light source transformations
 - Precompute the shadows and store them into a database
5. Shadow Z-Buffer
 - Integrate shadow computation into a Z-Buffer renderer

1. Shadows onto the ground plane

- ❑ Developed by James Blinn in 1988
- ❑ Features:
 - Restricted to single object scenes throwing shadows on a flat ground plane, or multi-object scenes where objects are isolated so as not to cast shadows on each other
 - Works quite well with a great number of light sources
- ❑ Two cases:
 1. Light source located at infinity
 2. Local light source
- ❑ Approach:
 - For each object polygon computes the shadow as a projection onto the plane $z=0$. For N sources will be N projections for each object polygon



1a. Light source at infinity

□ Problem:

- Consider 2 points: (1) the light source (x_l, y_l, z_l) , and (2) the point onto the object (x_p, y_p, z_p)
- Should compute the shaded point $S(x_s, y_s, z_s)$, that is in fact $S(x_s, y_s, 0)$

□ Method:

The point-vector equation of a line is:

$$S = P - \alpha L$$

on the ground plane $z_s = 0$

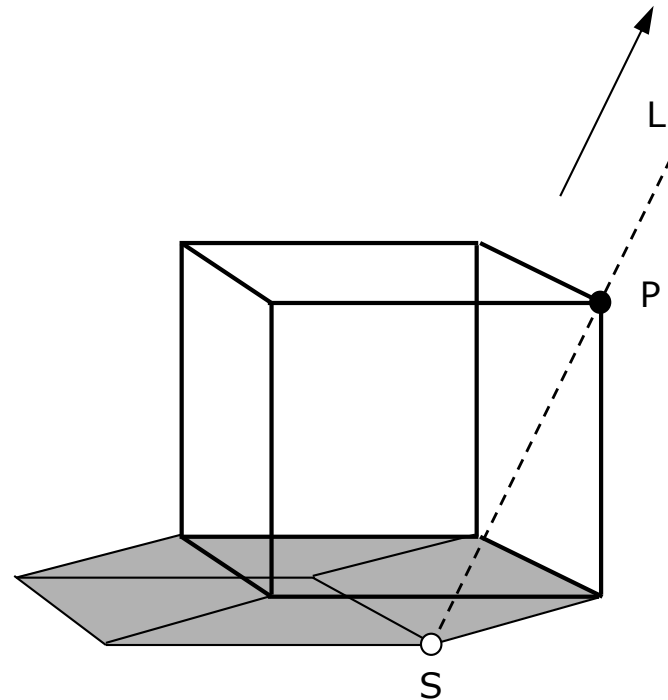
$$0 = z_p - \alpha z_l$$

$$\alpha = z_p / z_l$$

and

$$x_s = x_p - (z_p / z_l) x_l$$

$$y_s = y_p - (z_p / z_l) y_l$$



Light source at infinity

The matrix based transformation is:

$$S = P M_S$$

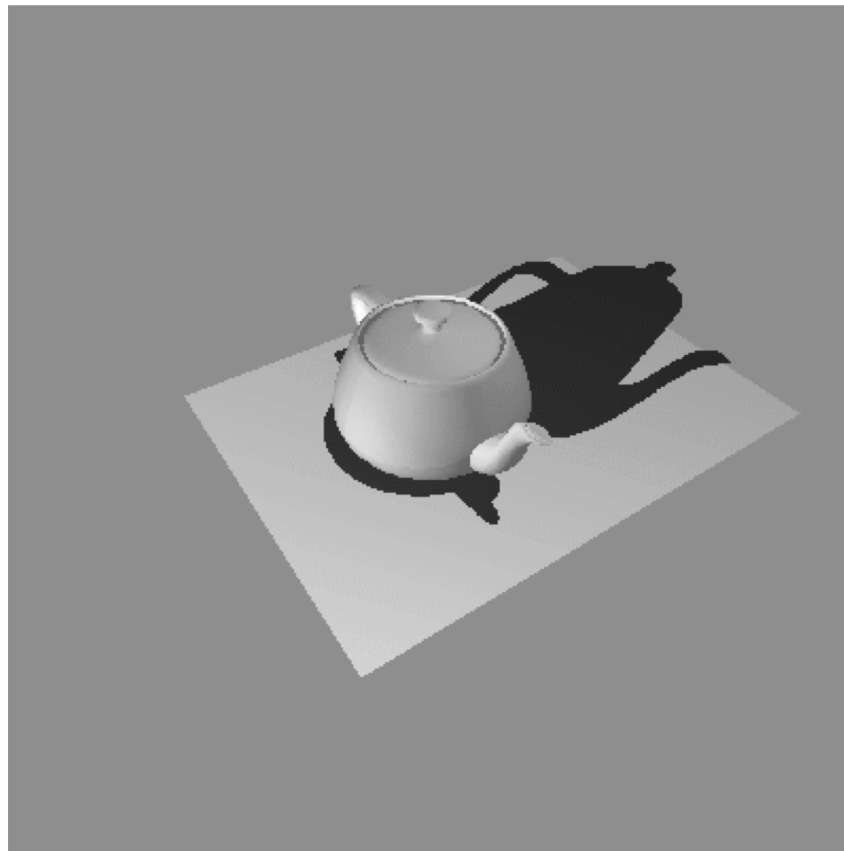
$$\begin{bmatrix} X_S & Y_S & 0 & 1 \end{bmatrix} = \begin{bmatrix} x_P & y_P & z_P & 1 \end{bmatrix} M_S$$

$$M_S = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ -x_l/z_l & -y_l/z_l & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Compute the vertices of the shadow polygons (projected polygons) and then fill in with the shadow color

Light source at infinity -example

- Example: shadows cast on the ground plane by two light sources located at infinity



1b. Local light source

- Shadow is computed by perspective projection from the location of the light source
- Consider 2 points: (1) the light source (x_l, y_l, z_l) , and (2) the point onto the object (x_p, y_p, z_p) . Should compute the shaded point $S(x_s, y_s, 0)$
- Method:

The point-vector equation of a line is:

$$S = P + \alpha (P - I)$$

on the ground plane $z_s = 0$

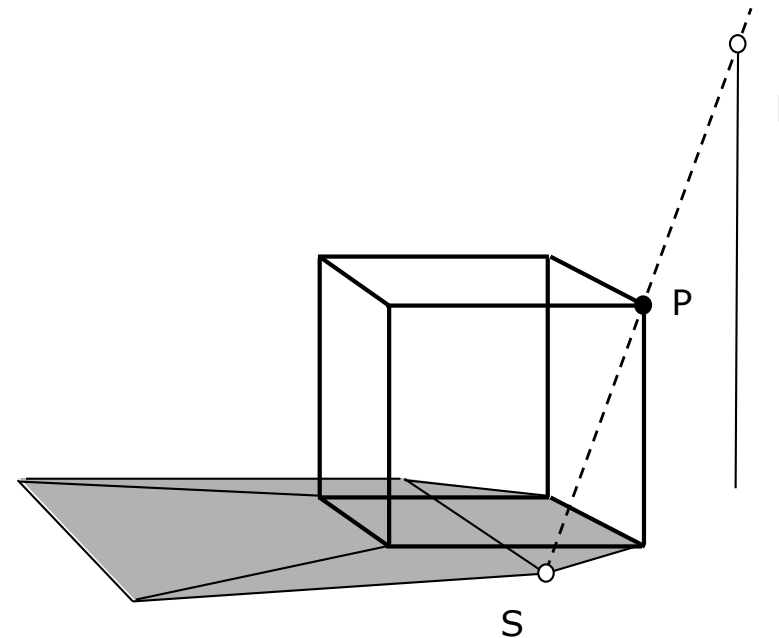
$$0 = z_p + \alpha (z_p - z_l)$$

$$\alpha = -z_p / (z_p - z_l)$$

and

$$x_s = (x_l z_p - x_p z_l) / (z_p - z_l)$$

$$y_s = (y_l z_p - y_p z_l) / (z_p - z_l)$$



Local light source

The matrix based transformation in homogeneous coordinates is:

$$S = P M_{SP}$$

$$\begin{bmatrix} X_S & Y_S & 0 & w \end{bmatrix} = \begin{bmatrix} x_P & y_P & z_P & 1 \end{bmatrix} M_S$$

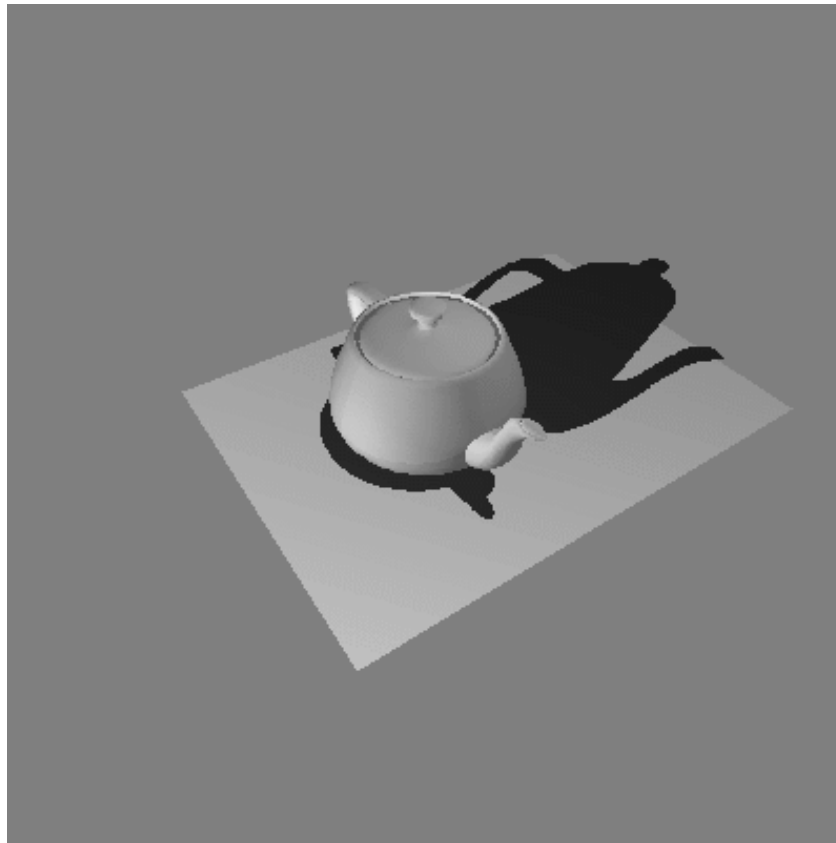
$$M_{SP} = \begin{bmatrix} -z_l & 0 & 0 & 0 \\ 0 & -z_l & 0 & 0 \\ x_l & y_l & 0 & 1 \\ 0 & 0 & 0 & -z_l \end{bmatrix}$$

$$x_S = X_S / w, \quad y_S = Y_S / w, \quad z_S = 0$$

- Compute the vertices of the shadow polygons (projected polygons) and then fill in with the shadow color

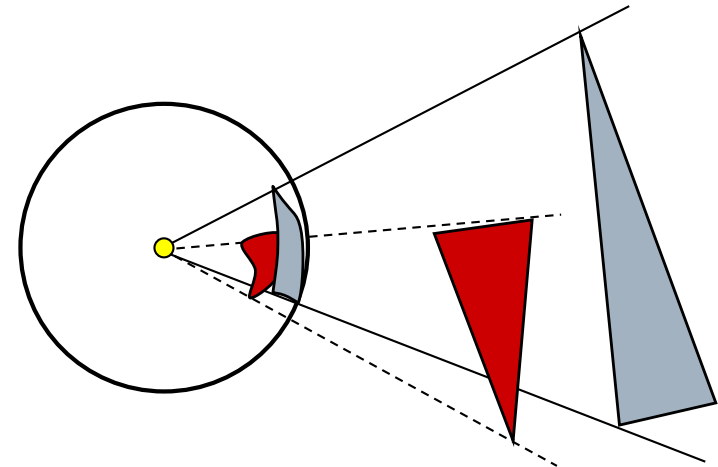
Local light source - example

- Example: shadows cast on the ground plane by two local light sources

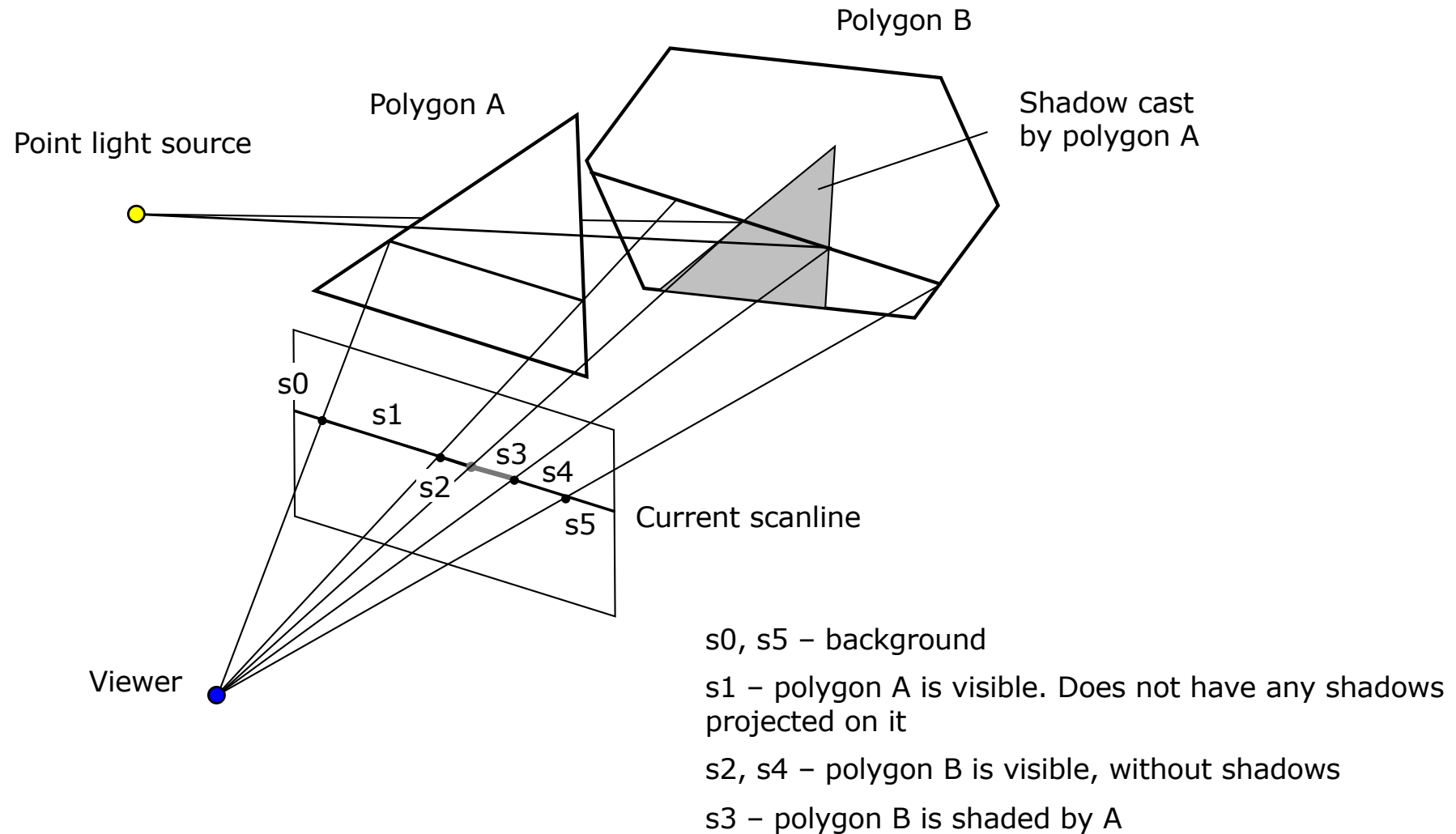


2. Shadow scanline

- ❑ Developed by Appel (1968), Bouknight and Kelley (1970)
- ❑ Scan line algorithm integrates shadow computation
- ❑ The algorithm works only with point light sources and generates hard edge shadows
- ❑ Requires a preprocessing phase
 - Builds a secondary data structure which links all polygons that may shadow a given polygon
 - Shadow pairs – a polygon together with the polygon that it can possibly shadow
 - Shadow pairs are detected by projecting all polygons onto a sphere centered at the light source
 - For a scene containing n polygons the possible projected shadows is $n(n-1)$



Shadow scanline



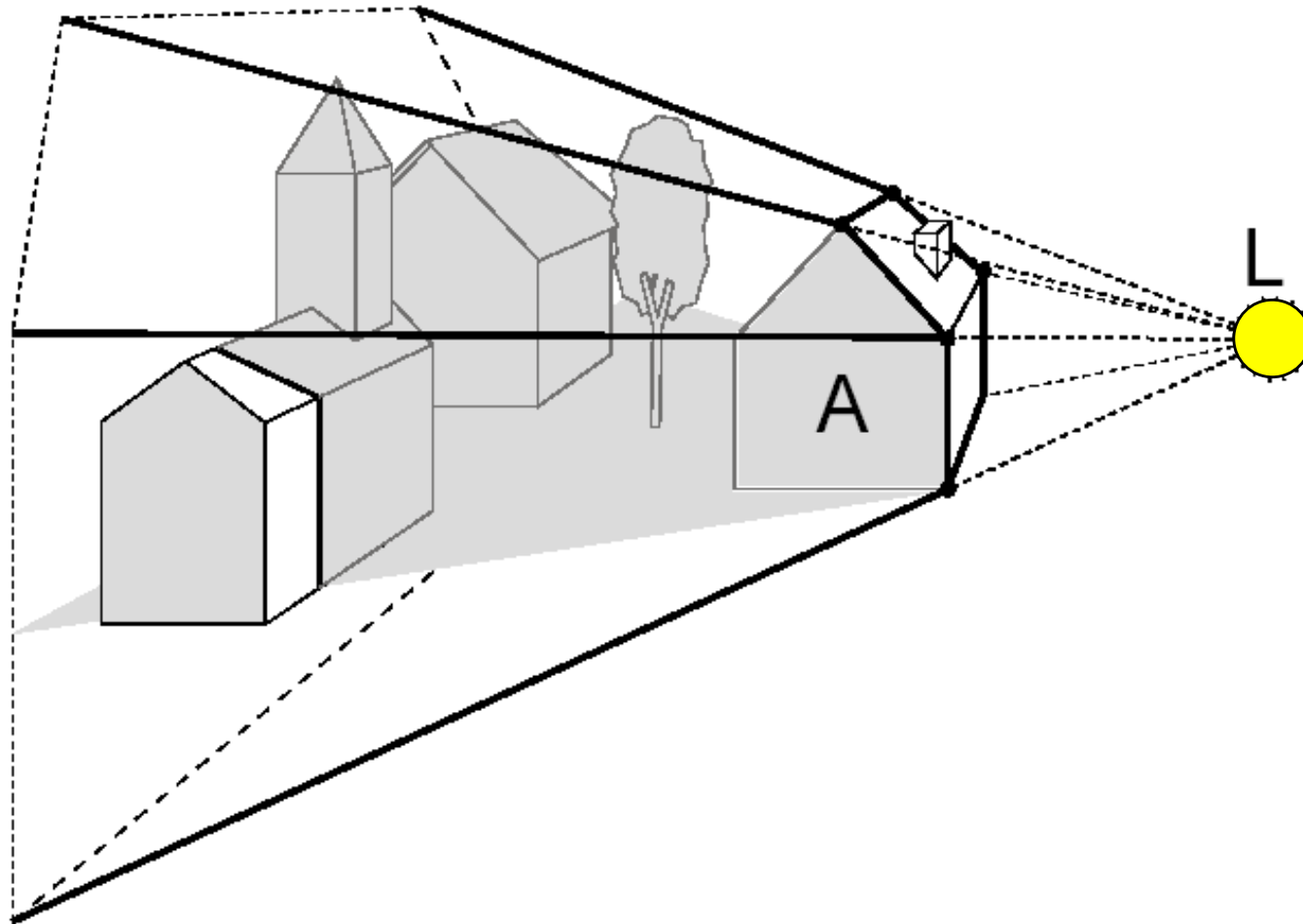
Shadow scanline algorithm

1. Processes secondary data structure simultaneously with a normal scan conversion process
2. Determine if any shadow fall on the polygon projected onto the current scanline
3. If shadow polygons does not exist the scanline algorithm proceeds as normal
4. If shadow polygon exists
 - Using the light source as a centre of projection, generates the shadow onto the plane of the current polygon
5. For each pixel on the current scan line, determines whether is in shadow or not. There are three possibilities:
 - a. The shadow polygon does not cover the generated scanline segment – render without shadows
 - b. Shadow polygon completely covers the visible scanline segment. The pixel intensity depends on the number of shadows projected on that pixel
 - c. A shadow polygon partially covers the visible scanline segment. The segment should be divided and the process is applied recursively until a clear solution is obtained

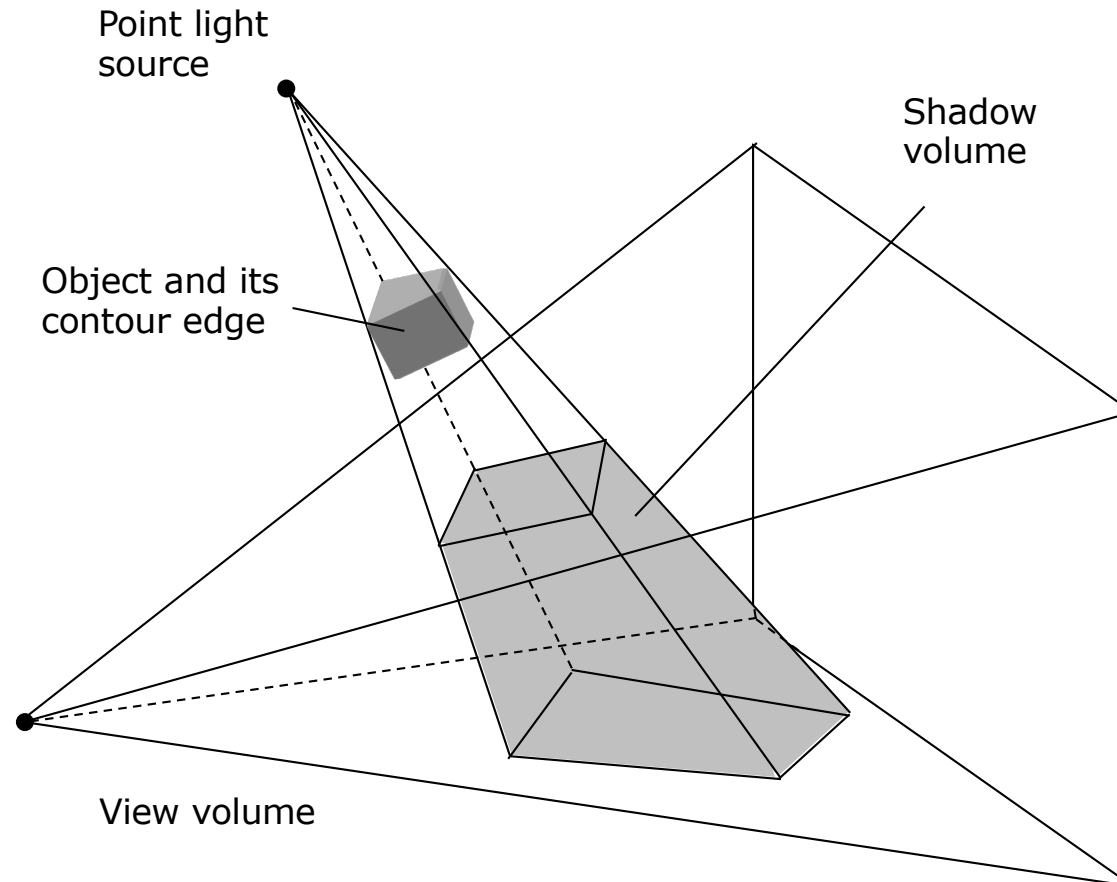
3. Shadow volumes

- ❑ Developed by Crow, F.C. (1977). Brotman, L.S. and Badler, N.I. (1984) use the idea to generate soft shadows (shadows produced by a distributed light source)
- ❑ Shadow volume – invisible volume of space swept out by the shadow of an object. It is the infinite volume defined by lines starting from a point light source and passing through the vertices of the object
- ❑ Finite shadow volume – intersection of the infinite volume with the view volume
- ❑ Shadow volume is cast by the contour edge of an object
- ❑ Shadow volume – the side faces are shadow polygons, that are stored in the graphics model. They are not rendered, but sorted, and clipped as the real polygons
- ❑ The algorithm is integrated with a depth priority hidden surface removal algorithm

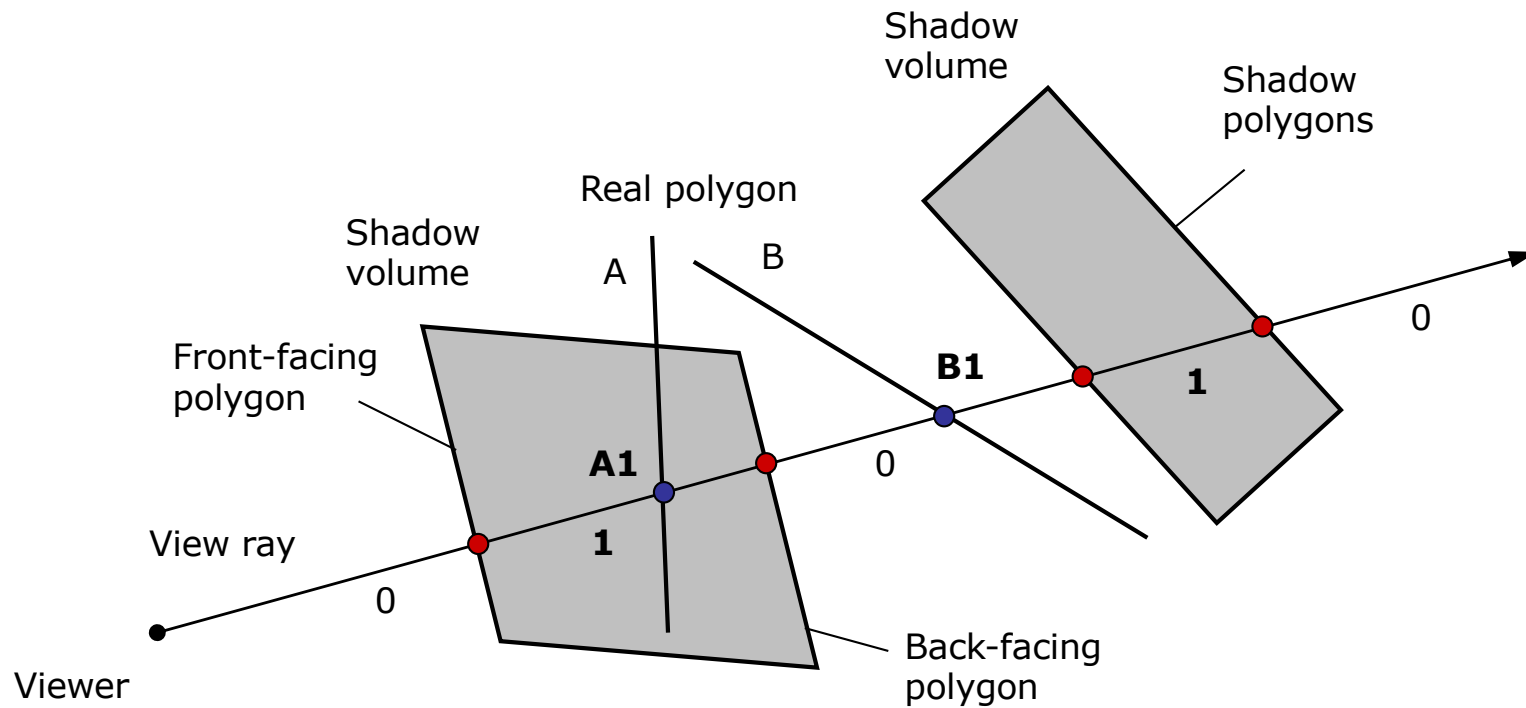
Shadow volumes



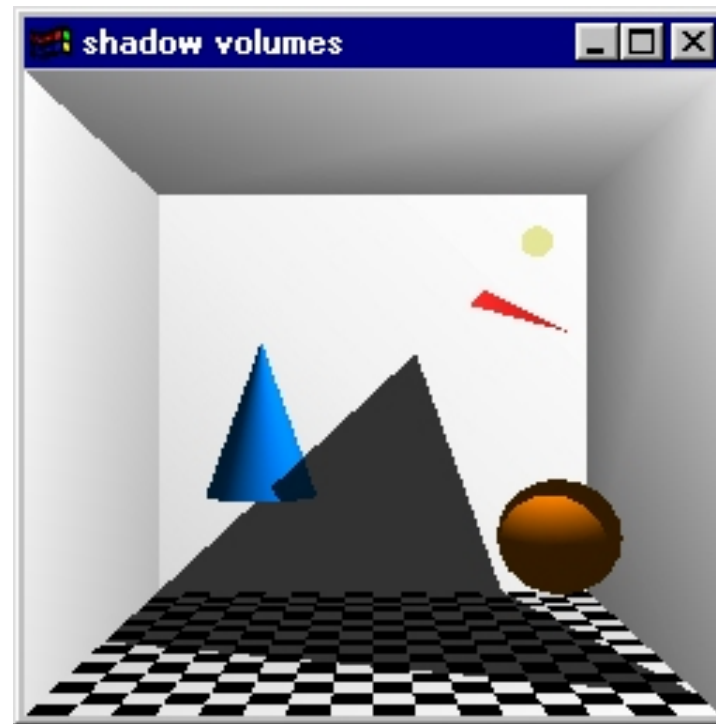
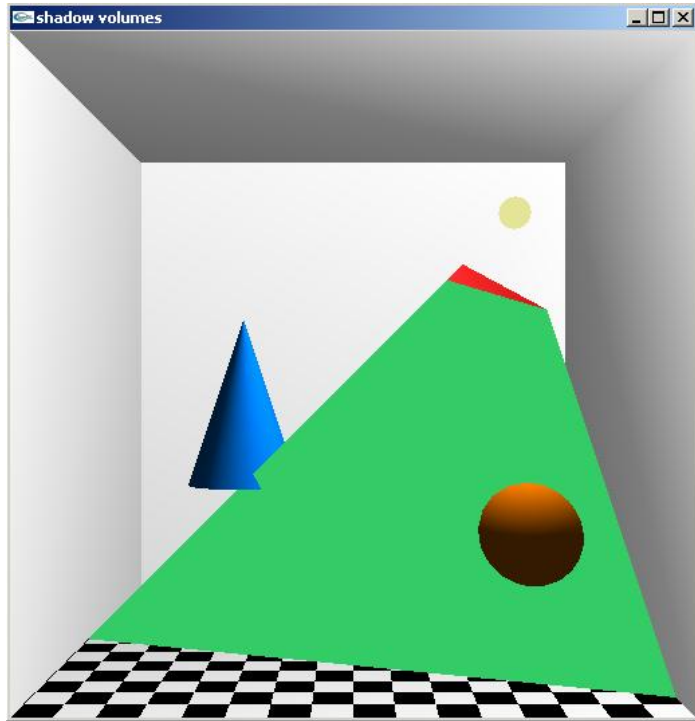
Shadow volumes



Shadow volumes



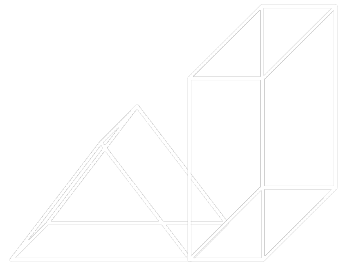
Shadow volumes - example



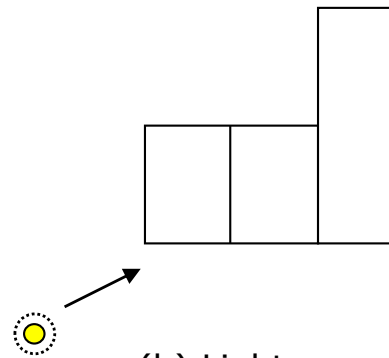
4. Shadows from transformations

- ❑ Developed by Atherton, Weiler and Greenberg, 1978
- ❑ Graphics model contains together the visible and non-visible polygons, from the light source location
- ❑ Based on the Weiler-Atherton algorithm for hidden surface removal (see the course “Hidden line surface removal - Part 1”), and the Weiler algorithm for polygon clipping (see the course “2D Clipping – Weiler Algorithm”)
- ❑ The scene is processed twice for hidden surface removal
 1. Using light source location as a view point, that produces the illuminated and shadow polygons
 2. Using any point of view, for normal visualization

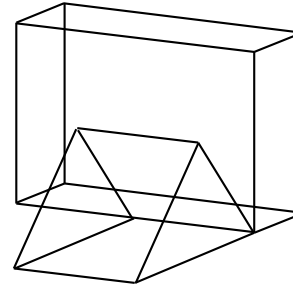
Shadows from transformations



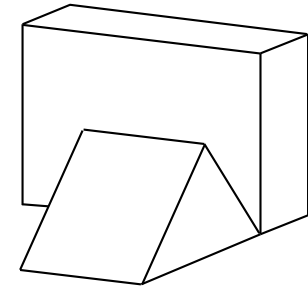
(a) Initial scene of objects



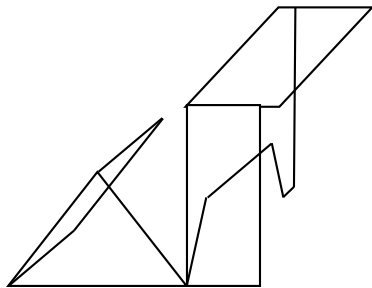
(b) Light source location



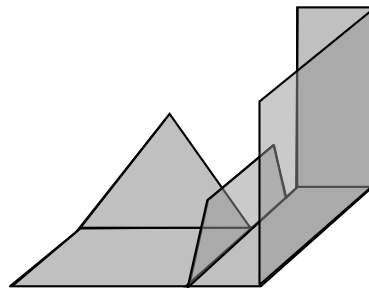
(c) The scene watched from the light source location



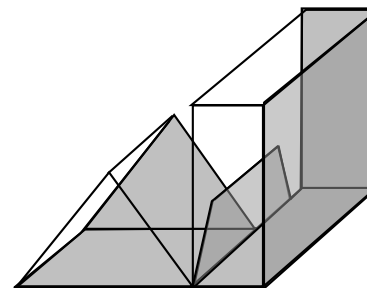
(d) Hidden surface visualization from the light source location



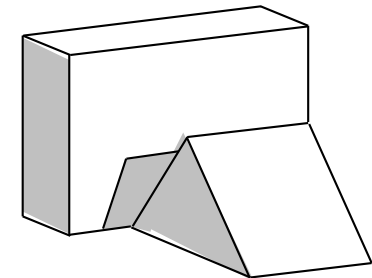
(e) Visible polygons: initial and clipped



(f) Non visible polygons



(g) Complete graphics model: visible (illuminated) and non visible (shaded) polygons



(h) Different visualization of the scene

5. Shadow Z-buffer

- ❑ Developed by Williams, and published in 1978

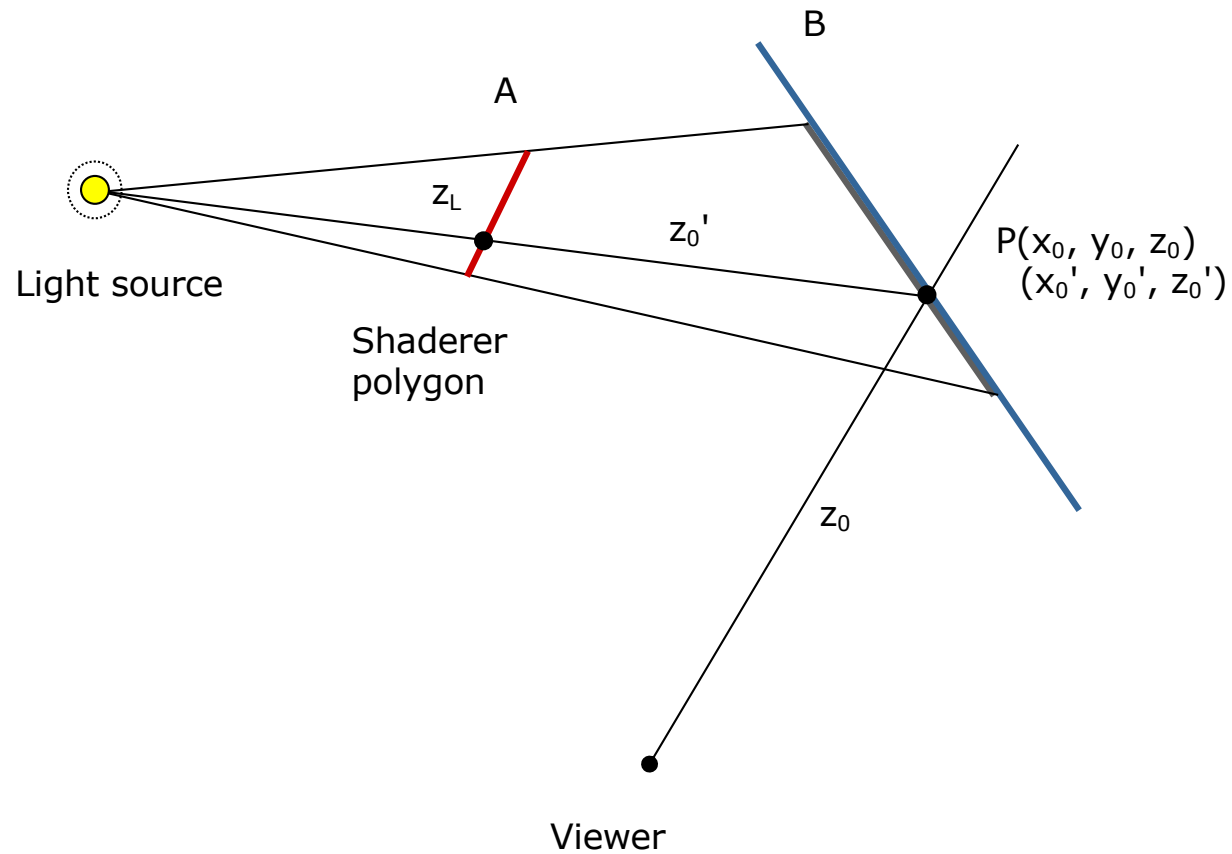
Williams, L: "Casting curved shadows on curved surfaces", Computer Graphics, 12(3), pp. 270-274, 1978

- ❑ The simplest approach to the shadow computation
- ❑ Can be easily integrated into Z-buffer based renderer
- ❑ The technique requires a shadow Z-buffer for each light source
- ❑ Two steps algorithm:
 1. The scene is rendered and depth information is stored into the shadow Z-buffer using the light source as a view point. No intensities are computed. The step computes the polygons that are visible to the light source
 2. Render the scene using a Z-buffer algorithm
 - a. If $P(x_0, y_0, z_0)$ is visible in viewer's centered coordinate system
 - b. Compute $P(x_0', y_0', z_0')$ in light source centered coordinate system
 - c. Compare the Z-buffer value with distance of P

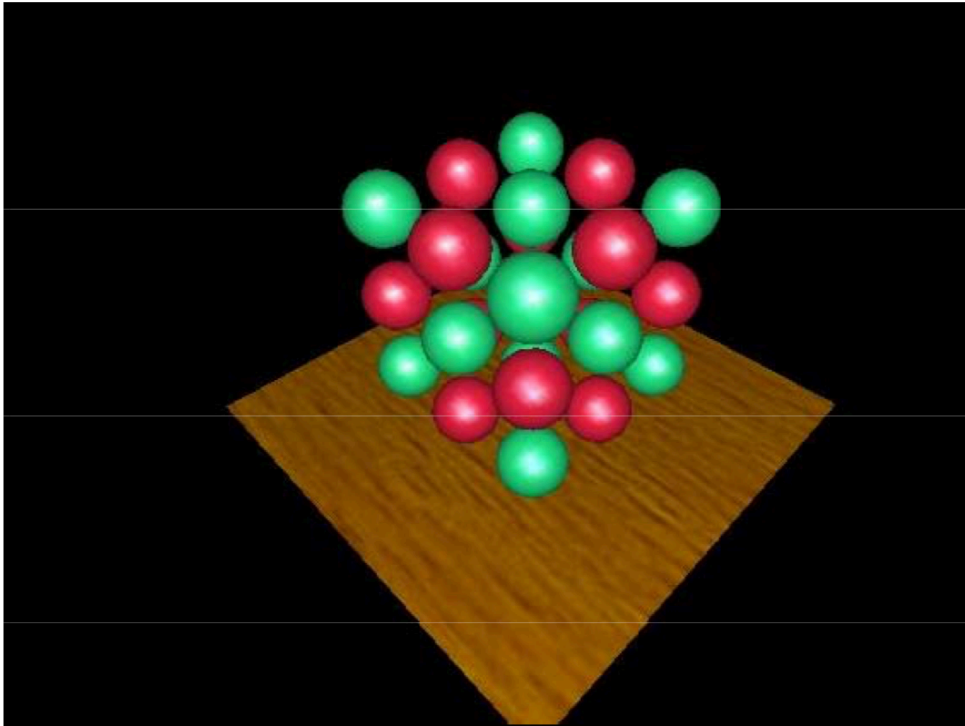
$z_L(x_0', y_0') < z_0'$ then P is in shadow

$z_L(x_0', y_0') \geq z_0'$ P is illuminated

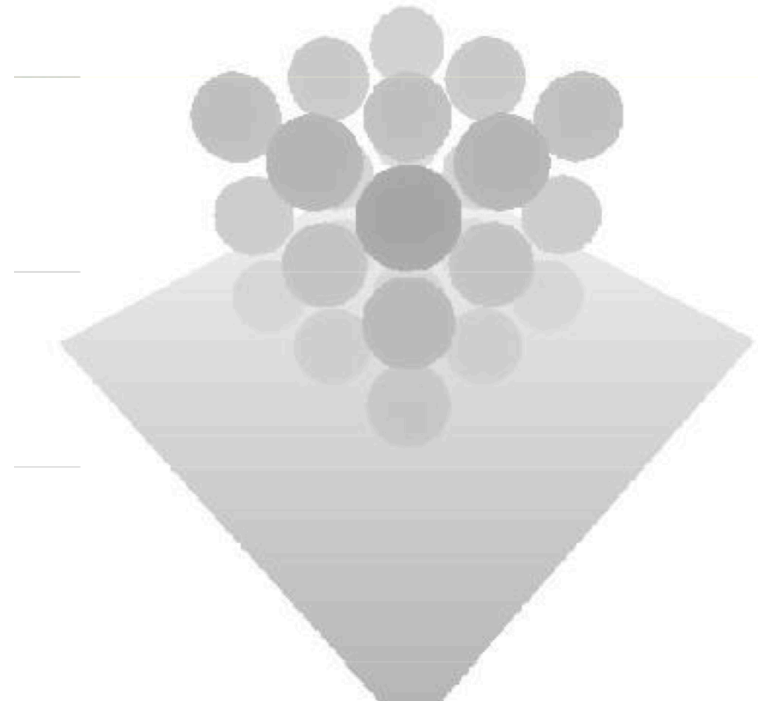
Shadow Z-buffer



Shadow Z-buffer – step 1

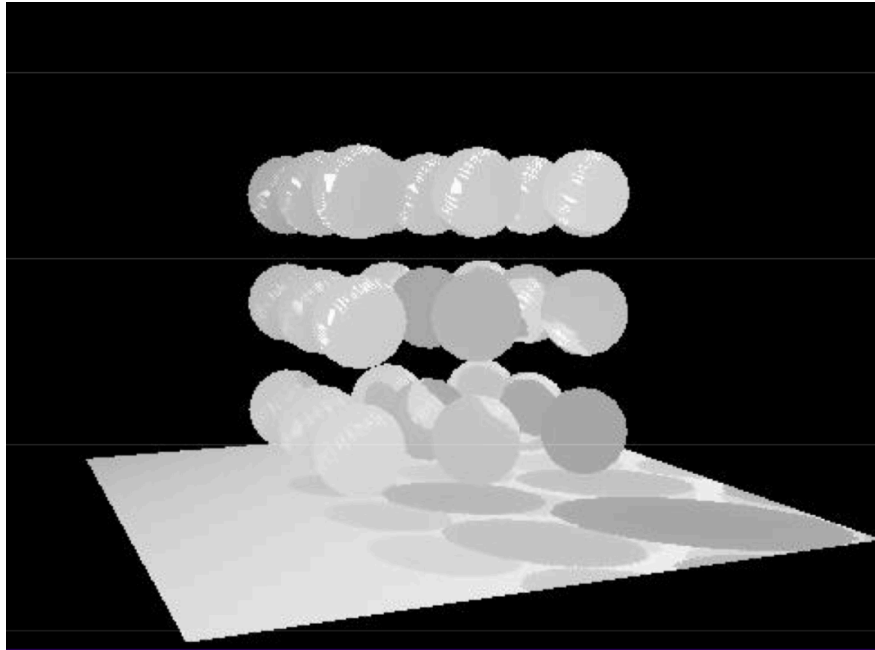


View from light

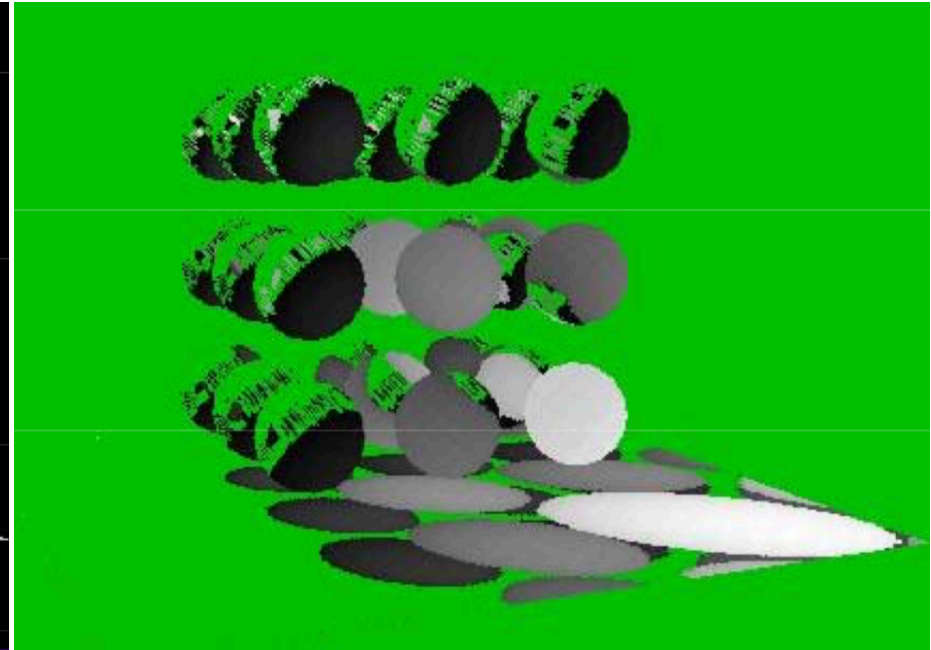


Depth buffer
(Shadow map)

Shadow Z-buffer – step 2

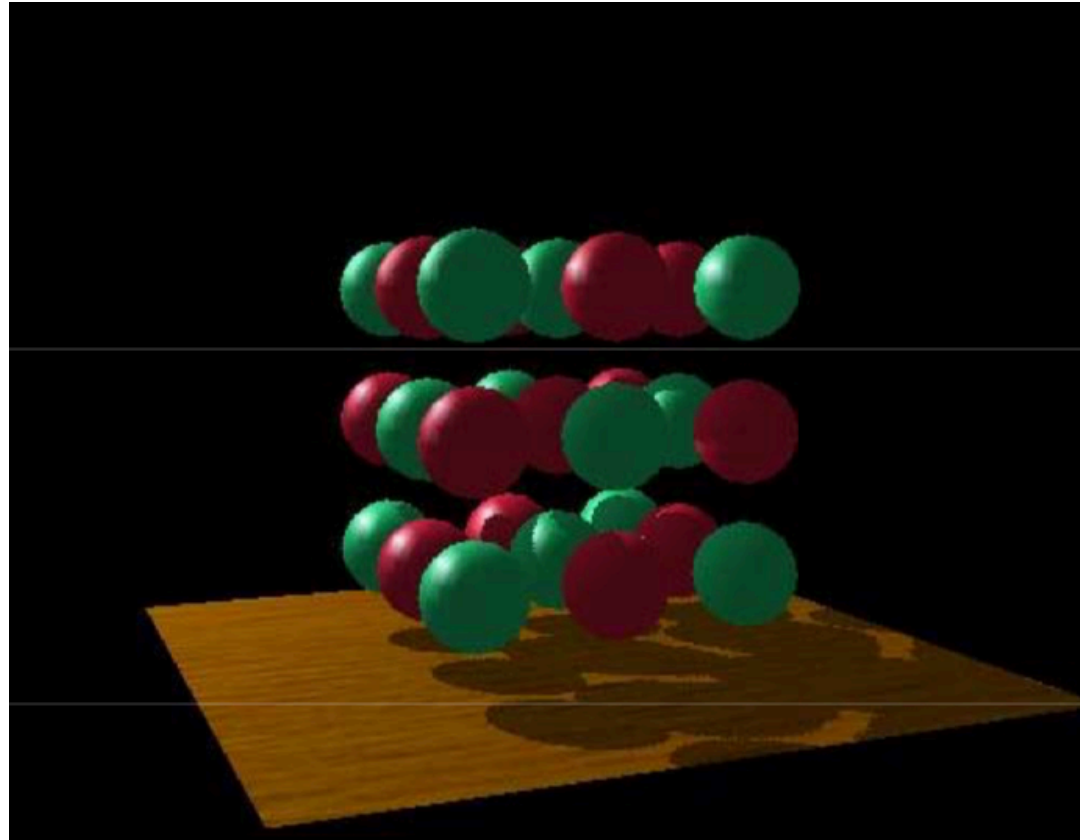


Visible surface depth



Non-green in shadow

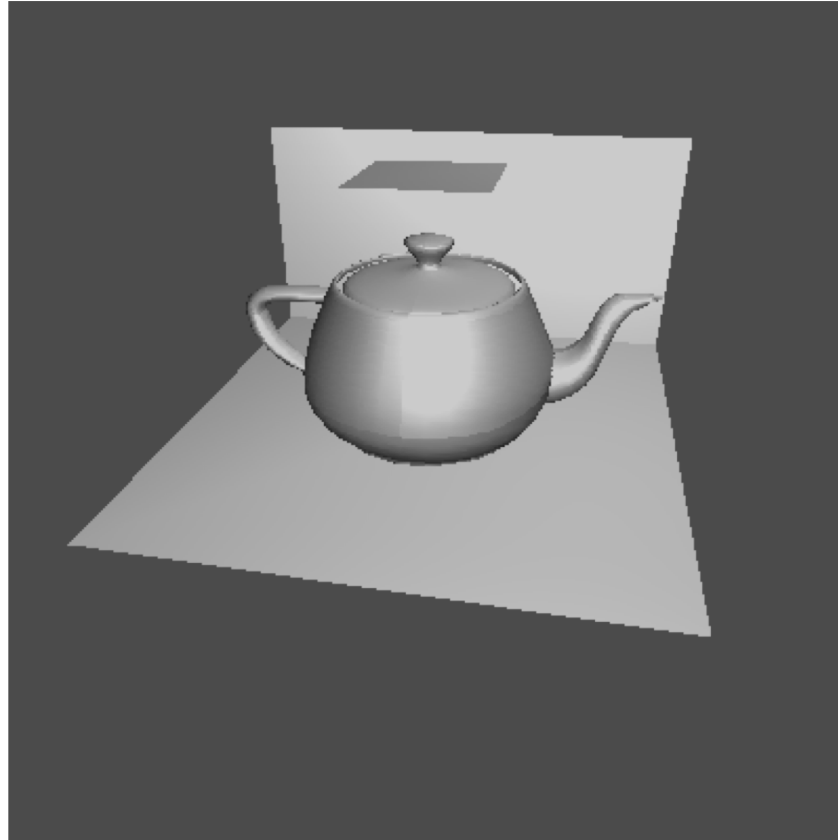
Shadow Z-buffer – final image



Shadow Z-buffer - issues

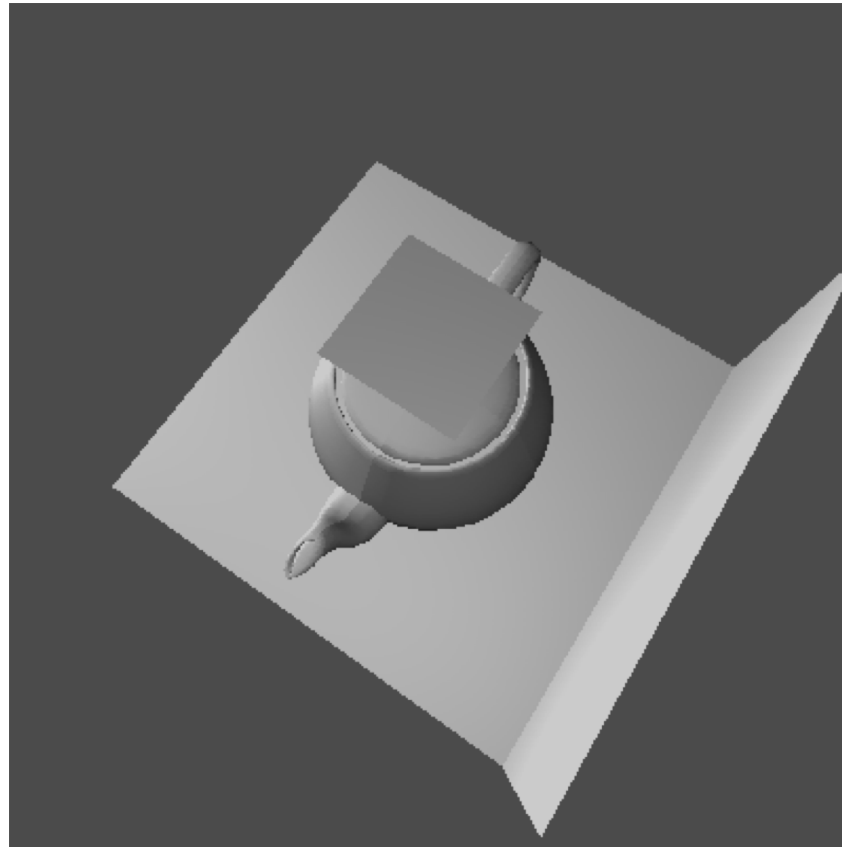
- Advantage:
 - can use the same shadow Z-buffers for many viewing locations
- High memory requirements
 - Shadow Z-buffer for each light source
 - Z-buffer for normal rendering
 - Image buffer
- Computation errors
 - For float point approximation a polygon could shadow itself
 - Solution: proximity projection – P' is considered closer to the light source than P
- Time and computation resources
 - Avoid computation of shadow Z-buffer in precalculation phase
 - Compute the depth in shadow Z-buffer in the normal rendering phase just for the visible points (i.e. P)

Shadow Z-buffer - examples



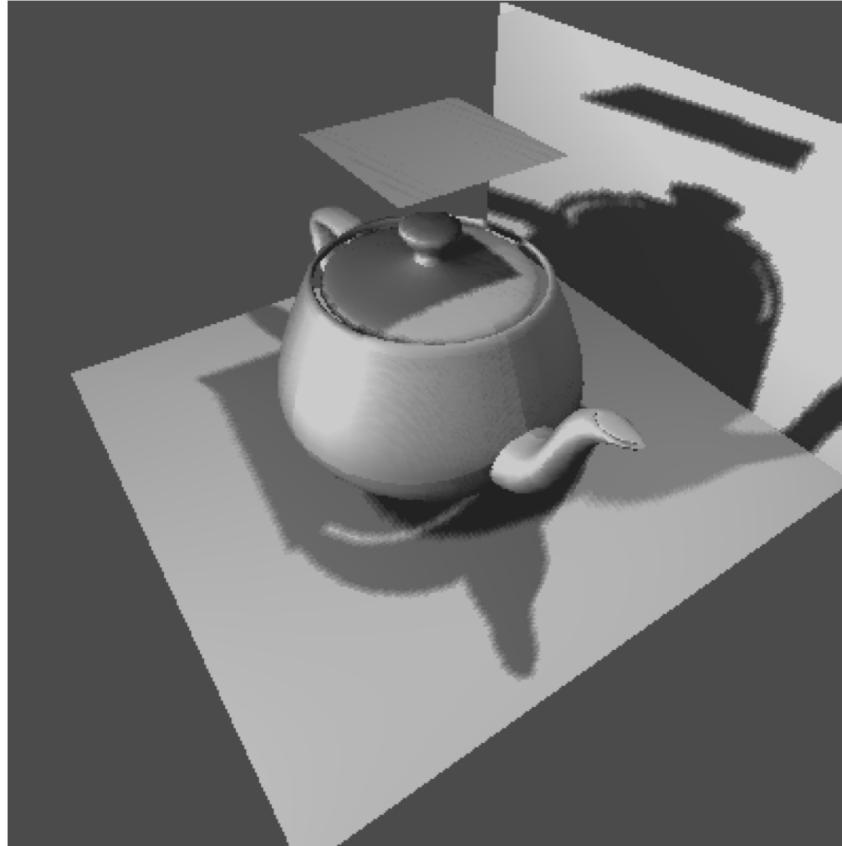
Light source 1

Shadow Z-buffer - examples



The object watched by the light source 2

Shadow Z-buffer - examples



Shadows generated by the two light sources

Examples given by Chris Bentley at <http://web.cs.wpi.edu/~matt/courses/cs563/talks/shadow/shadow.html>

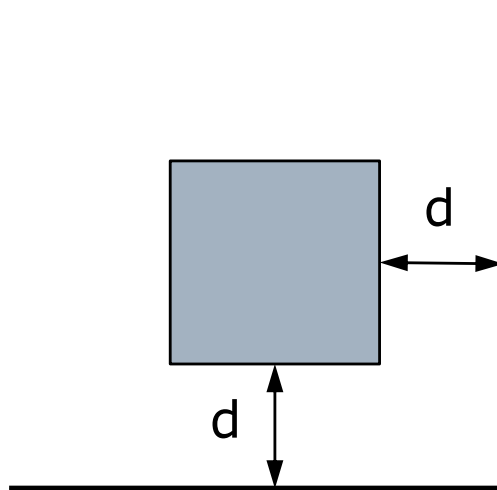
References:

1. Gregory S. Johnson, William R. Mark, Christopher A. Burns: The Irregular Z-Buffer and its Application to Shadow Mapping.
<http://www.cs.utexas.edu/ftp/pub/techreports/tr04-09.pdf> (2004)
2. Grant C.: Visibility Algorithms in Image Synthesis. PhD thesis, University of California (1992).
3. Hasenfratz J.-M., Lapierre M., Holzschuch N., Sillion F.: A Survey of Real-Time Soft Shadows Algorithms. Computer Graphics Forum, 22, 4 (2003). State-of-the-Art Report.
4. Razvan Surdulescu: Cg Shadow Volumes (2003),
<http://www.gamedev.net/reference/articles/article1990.asp>

Questions and problems

1. What cases of objects and light sources give soft edge or hard edge shadows?
2. How it could be created soft shadows by point light sources?
3. Exemplify and explain the casting of shadows onto the ground plane and the vertical plane if the light source is located at infinity, for a cube (edge = $2a$, distance $d = a$) like in the following figure 1:

Figure 1.



Questions and problems

4. Exemplify and explain how are computed the shadow polygons through the Shadow Scan Line algorithm for the object in figure 2, which consists of three opaque triangles as components (red, blue, green). The light source and the viewer are located at infinity, each one on the specified line.

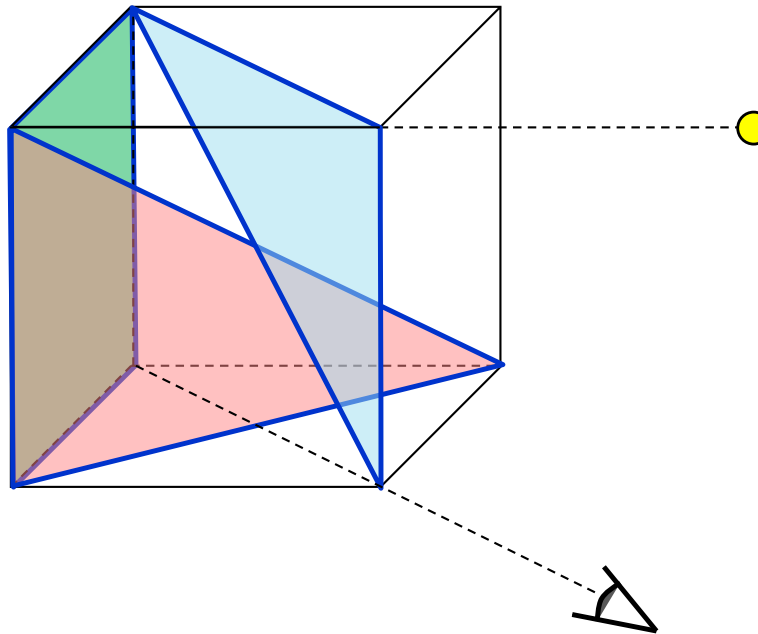
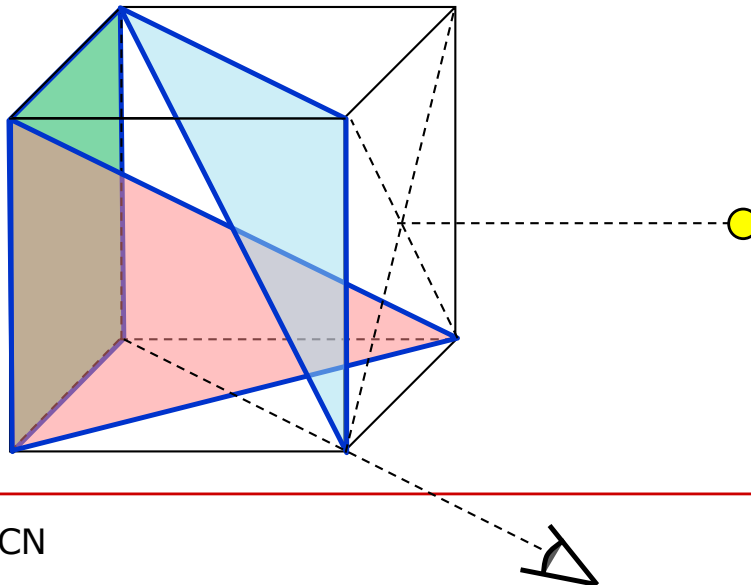


Figure 2.

Questions and problems

5. Exemplify and explain the Shadow Scan Line algorithm for the object in figure 2, which consists of three opaque triangles as components (red, blue, green). The light source and the viewer are located at infinity, each one on the specified line.
6. Exemplify and explain the Shadow Volume algorithm for the three triangles in figure 3. Explain the shadow computation for one viewing ray. Viewer at infinity and light source at a limited distance out of the virtual cube (e.g. distance = cube edge length)

Figure 3.



Questions and problems

7. Exemplify and explain the Shadow by Transformation algorithm for the object in figure 4. The light source and the viewer are located at infinity, each one on the specified line.

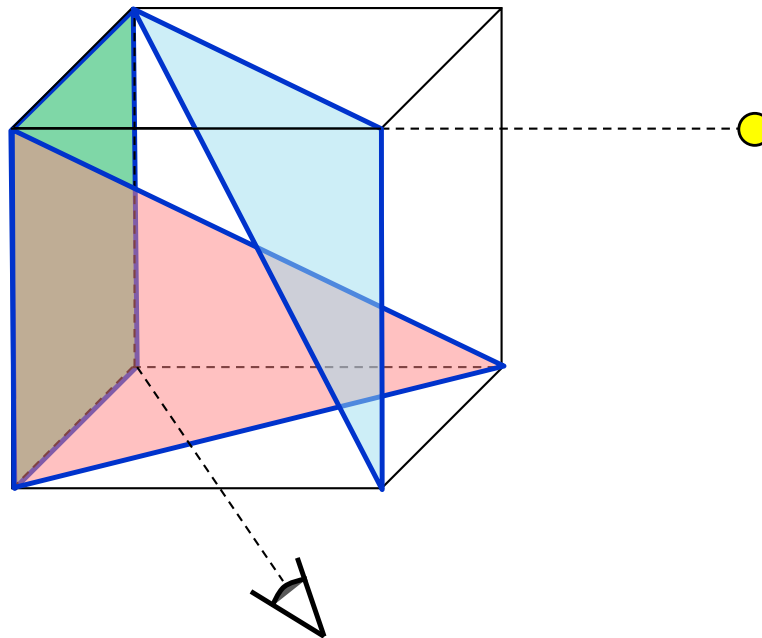


Figure 4.

Questions and problems

8. Exemplify and explain the Shadow z-Buffer algorithm for the object in figure 5, which consists of three opaque triangles as components (red, blue, green). The light source and the viewer are located at infinity, each one on the specified line.

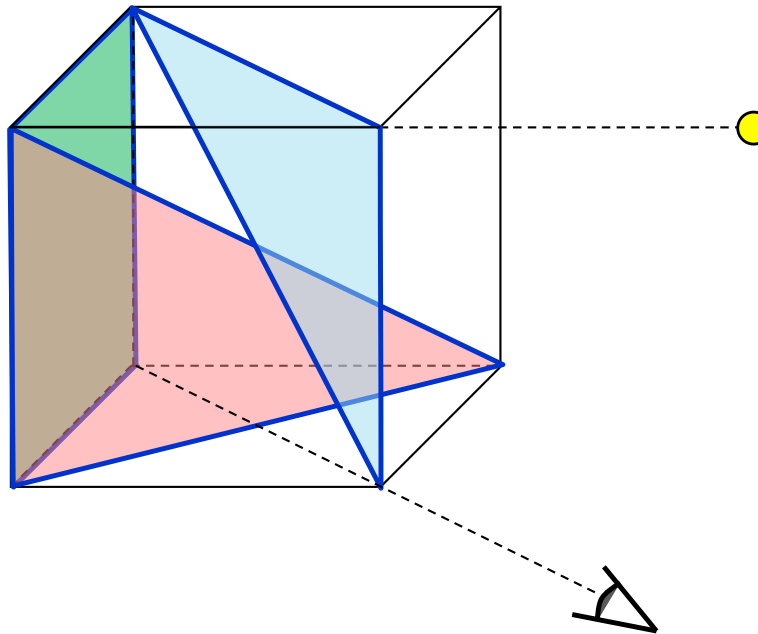


Figure 5.